

Resume-to-Job Matching using Feature Extraction, Cosine Similarity, and Principal Component Analysis

Jacob Curtis, Mohd Aftab Shaikh, Prajwal Gangadhar Melinamane

Northeastern University

Portland, Maine

Abstract

This paper presents a data-driven system for resume matching with existent occupations, using structured feature extraction, cosine similarity, and Principal Component Analysis (PCA). The system transforms unstructured resume text into numerical feature vectors representing skill sets applicable to available occupations. These vectors are compared to job descriptions to generate ranked matches. Additionally, PCA is applied to explore the structure of the feature space and enable visualization.

1 Introduction

The process of matching job candidates to relevant employment opportunities is a critical task in modern labor markets. As organizations receive increasing numbers of applications, manual resume screening becomes inefficient, time-consuming, and prone to bias. Automated systems have emerged to address these challenges, but many rely on simplistic keyword matching approaches that fail to capture deeper relationships between candidate qualifications and job requirements.

Resumes are inherently unstructured documents that vary widely in format, language, and level of detail. Candidates often describe similar skills using different terminology, making direct comparison difficult. This variability introduces challenges for automated systems and highlights the need for structured representations of textual data.

This paper proposes a system that converts resume text into structured feature vectors representing skill categories. These vectors enable mathematical comparison using cosine similarity, a widely used metric in information retrieval. Additionally, Principal Component Analysis (PCA) is applied to reduce dimensionality and visualize relationships within the data.

The goal of this work is to develop a scalable system for resume matching while also considering trade-offs introduced by dimensionality reduction techniques.

2 Related Work

Automated resume screening has been extensively studied in the fields of machine learning and natural language processing. Early approaches relied on keyword matching and rule-based systems, which are simple but lack robustness.

Vector Space Models (VSM) introduces a more structured approach by representing documents as vectors of term frequencies. Techniques such as TF-IDF weighting has improved performance by reducing the influence of common terms. The two proposed methods are common in many resume-matching related activities.

3 System Overview

The proposed system consists of several stages:

1. Resume ingestion
2. Text extraction

3. Preprocessing
4. Feature extraction
5. Feature scaling
6. Similarity computation
7. Dimensionality reduction
8. Visualization and ranking

4 Methodology

4.1 Text Extraction and Cleaning

The first step in the system involves extracting raw textual data from resumes, which are typically stored in PDF format. Because resumes are inherently unstructured and often contain varying layouts, fonts, and formatting styles, extracting usable text requires a reliable parsing method. In this project, the Python library PDFPlumber is used to read each resume file and extract text from individual pages. The extracted text from all pages is then concatenated into a single continuous string representing the full resume.

For mathematical representation of this extraction method, let the extracted text be denoted as T , where T represents the complete textual content of a resume. This raw text often contains inconsistencies such as capitalization differences, punctuation, and irregular spacing. To standardize the data and improve the reliability of downstream processing, a cleaning function is applied.

The cleaning process consists of three main operations. First, all characters are converted to lowercase to ensure that keyword matching is case-insensitive. Then, non-alphanumeric characters such as punctuation and symbols are removed, as they do not contribute mean-

ingfully to skill identification. Lastly, multiple whitespace characters are reduced to single spaces to normalize formatting.

Mathematically, the cleaned text can be represented as:

$$\tilde{T} = f(T)$$

where f is the cleaning function.

This step is critical because the accuracy of feature extraction depends heavily on consistent text formatting. Without normalization, the same word could be represented in multiple ways, leading to incorrect or missed matches. By standardizing the text, the system ensures that keyword detection operates on a uniform representation.

4.2 Feature Extraction

After text cleaning, the system converts unstructured resume text into a structured numerical representation using standardized occupational skill dimensions derived from the O*NET database. This transformation allows resumes and job roles to be embedded within a common vector space, enabling direct mathematical comparison.

Let $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$ denote the set of selected O*NET skill dimensions, where n is the number of features retained after preprocessing and variance filtering. Each resume is mapped into an n -dimensional feature space:

$$\mathbf{x} \in \mathbb{R}^n, \quad \mathbf{x} = [x_1, x_2, \dots, x_n]$$

where each component x_i corresponds to the extracted value of skill s_i .

To compute x_i , each skill s_i is associated with a set of keywords $K_i = \{k_{i1}, k_{i2}, \dots, k_{im_i}\}$. These keywords are derived from the skill name. The cleaned resume text is denoted by $\tilde{T}$, which is treated as a sequence of tokens.

We define an indicator function:

$$\mathbb{I}(k, \tilde{T}) = \begin{cases} 1, & \text{if keyword } k \text{ appears in } \tilde{T} \\ 0, & \text{otherwise} \end{cases}$$

However, since frequency information is retained, the feature value is computed using a formulation based on counts:

$$x_i = \sum_{j=1}^{m_i} \text{count}(k_{ij}, \tilde{T})$$

where $\text{count}(k_{ij}, \tilde{T})$ denotes the number of occurrences of keyword k_{ij} within the cleaned text.

Equivalently, the feature vector can be interpreted as a projection of the text onto the skill space:

$$\mathbf{x} = \Phi(\tilde{T})$$

where $\Phi : \mathcal{T} \rightarrow \mathbb{R}^n$ is the feature extraction mapping from text space $\mathcal{T}$ to the numerical skill space.

For a collection of M job roles, the dataset can be represented as a matrix:

$$X \in \mathbb{R}^{M \times n}$$

where each row $\mathbf{x}_i$ corresponds to a job's skill vector derived from O*NET data. Similarly, each resume is represented as a vector $\mathbf{x}_{\text{res}} \in \mathbb{R}^n$, ensuring that both resumes and job roles lie in the same feature space.

An important property of this representation is sparsity. Since most resumes do not explicitly mention all possible skills, many components of $\mathbf{x}$ are zero:

$$\|\mathbf{x}\|_0 \ll n$$

where $\|\cdot\|_0$ denotes the number of non-zero elements. This sparsity reflects the selective nature of skill expression in resumes and therefore contains implications for similarity computation, as only a subset of dimensions contributes to the overall alignment.

Additionally, the dimensionality n can be relatively large due to the breadth of O*NET skill definitions. To reduce redundancy, feature selection is applied by removing low-variance dimensions. Formally, this can be expressed as selecting a subset $\mathcal{S}' \subset \mathcal{S}$ such that:

$$\text{Var}(s_i) > \tau$$

for some threshold τ . This results in a reduced feature space:

$$\mathbf{x} \in \mathbb{R}^{n'}, \quad n' < n$$

where only informative features are retained.

This feature extraction process provides a structured embedding of textual data into a numerical vector space that coincides with the structured O*NET occupation vector space. The resulting representation enables the application of linear algebra techniques, such as cosine similarity and dimensionality reduction, in subsequent stages of the system.

4.3 Feature Scaling

Once feature vectors are generated, it is necessary to normalize the values to ensure that all features contribute equally to similarity computations.

This is done by utilizing Python’s MinMax Scaler function. This is an important method in the similarity computation as raw keyword counts can vary significantly across features and resumes. For instance, commonly mentioned skills may have higher counts simply due to verbosity, for example an applicant utilizing lengthy terminology instead of raw skillsets. This common application of detail in resumes may not lead to important components for

cosine similarity.

Min-Max scaling is applied to each feature. This transformation rescales feature values to the range $[0, 1]$, allowing for the relative differences.

The scaling function is defined as:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

where $x_{\min}$ and $x_{\max}$ are the minimum and maximum values of the feature across the dataset.

Let the job dataset be represented as a matrix $X \in \mathbb{R}^{M \times n}$, where M is the number of job roles. The scaler is fit on this dataset to learn the appropriate transformation parameters. The same transformation is then applied to resume feature vectors.

One other purpose of scaling is to prevent features with larger numerical ranges from dominating similarity calculations. Without scaling, features with higher counts would disproportionately influence cosine similarity, and thus, potentially leading to biased results.

The scaling in python is computed by StandardScaler from Python's Scikit-Learn package. Please see the figure below for scaling of features.

```
scaler = MinMaxScaler()  
X_scaled = scaler.fit_transform(X_jobs)
```

Figure 1: StandardScaler Representation in Python

By normalizing all features to the same range, the system allows for similarity comparisons that are based on relative patterns rather than absolute magnitudes. This step is essential for maintaining standardized 'fairness' and consistency in the matching process.

4.4 Cosine Similarity

After scaling, similarity between resumes and job roles is computed using cosine similarity. This metric measures the cosine of the angle between two vectors, capturing how closely they align in the feature space.

Given two vectors $\mathbf{x}$ and $\mathbf{y}$, cosine similarity is defined as:

$$\text{sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

This measure ranges from 0 to 1 for non-negative vectors, where values closer to 1 indicate stronger similarity.

In this system, the resume vector is compared to each job vector in the dataset. Let $\mathbf{x}_{\text{res}}$ represent the scaled resume vector and $\mathbf{x}_i$ represent the i -th job vector. The similarity score is computed as:

$$s_i = \text{sim}(\mathbf{x}_{\text{res}}, \mathbf{x}_i)$$

The system then ranks job roles based on these similarity scores and selects the top matches.

```
sim_raw = cosine_similarity(X_res_scaled, X_scaled)[0]
top_idx_raw = sim_raw.argsort()[-3:][::-1]

print("\nTop Matches (NO PCA):")
for i in top_idx_raw:
    print(f" - {job_roles.iloc[i]} (score: {sim_raw[i]:.4f})")

X_res_pca = pca.transform(X_res_scaled)
sim_pca = cosine_similarity(X_res_pca, X_pca_jobs)[0]
top_idx_pca = sim_pca.argsort()[-3:][::-1]
```

Figure 2: Cosine Similarity Representation in Python

Cosine similarity is particularly suited for this task because it focuses on the direction of vectors rather than their magnitude. This means that two resumes with similar skill distributions will be considered similar even if one contains more total words than the other.

Cosine Similarity thus is a proper method for calculating the similarity between two non-zero vectors. The two non-zero vectors, original feature vector from the dataset. The other non-zero vector is the transformed vector from the matching function. The formula shown previously,

$$s_i = \text{sim}(\mathbf{x}_{\text{res}}, \mathbf{x}_i)$$

is the cosine of the angle proposed by the two vectors.

Formally represented as,

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

4.5 Principal Component Analysis

Principal Component Analysis (PCA) is applied to reduce the dimensionality of the feature space and uncover latent structure within the data. High-dimensional feature spaces can be difficult to interpret and visualize, especially when many features are correlated.

PCA addresses this by projecting the data onto a new set of orthogonal axes, known as principal components, which capture the directions of maximum variance. If X is the scaled data matrix, PCA computes a transformation matrix W such that:

$$Z = XW$$

where Z is the transformed data in a lower-dimensional space.

In this project, the data is reduced to three principal components, allowing for 3D vi-

sualization. Each job role and resume is represented as a point in this reduced space. The overall resume will be demonstrated with a larger point in this space, while the occupations will be smaller and a fraction of the resume size. This allows for ease of understanding with this program's visuals.

The resume vector is also projected using the same transformation:

$$\mathbf{z}_{\text{res}} = \mathbf{x}_{\text{res}}W$$

Cosine similarity is then recomputed in the PCA space to analyze how dimensionality reduction affects matching.

Below is the PCA method from Python's Skit-learn package. PCA is loaded with 3 components for representation in the graph.

```
pca = PCA(n_components=3)
X_pca_jobs = pca.fit_transform(X_scaled)

print("\nExplained Variance (PCA):", pca.explained_variance_ratio_)
```

Figure 3: PCA Representation in Python

The recomputation for the cosine similarity demonstrated in python:

```
X_res_pca = pca.transform(X_res_scaled)
sim_pca = cosine_similarity(X_res_pca, X_pca_jobs)[0]
top_idx_pca = sim_pca.argsort()[-3:][::-1]
```

Figure 4: PCA with Cosine Similarity

PCA serves two primary purposes in this system. First, it reduces complexity, making it

easier to analyze relationships between job roles. Second, it enables visualization, providing an intuitive geometric interpretation of similarity.

However, PCA also introduces trade-offs. While it captures the most significant variance in the data, it may discard information that is important for accurate matching. As a result, similarity scores computed in the reduced space may differ from those in the original feature space.

Despite these limitations, PCA provides valuable insights into the structure of the data and helps illustrate how resumes and job roles cluster based on shared skill patterns.

5 Data Summary

The dataset used in this project is derived from the O*NET Skills database and consists of structured representations of occupational skill requirements. Unlike traditional datasets containing demographic attributes such as age or gender, this dataset captures skill-based characteristics of occupations. Each observation corresponds to a job role, and each skill value represents the importance level of a specific skill.

The underlying O*NET skill and occupational data are collected through ongoing, nationwide surveys of job incumbents and occupational experts, and this is managed by the National Center for O*NET Development. The data is continuously updated using random sampling methods to reflect current labor market conditions. As a result, the dataset provides a reliable and standardized representation of workforce skill requirements.

After filtering for the “Level” scale and restricting the dataset to a subset of construction related occupations, the data was transformed into a matrix where rows correspond to job roles and columns correspond to skill features. Missing values were replaced with zero, and low-variance features were removed to reduce noise. Overall, this resulted in 30 rows of filtered ONET job codes, allowing for a concentrated analysis on relative occupations to that of the State of Maine. To be clear, the skill-set database consists of 62,000 rows of

occupational data. To capture signal in this dataset, filtration was therefore considered.

The descriptive statistical analysis reveals that several skills have consistently high importance across occupations. The top mean values of the features include Coordination (3.33), Critical Thinking (3.33), Active Listening (3.14), Monitoring (3.11), and Reading Comprehension (3.11).

In contrast, features such as Installation (1.34), Repairing (1.21), Equipment Maintenance (1.16), Management of Financial Resources (1.13), Operations Analysis (1.12), and Science (1.08) exhibit higher standard deviation. This suggests that these skills are more specialized, with their importance varying significantly depending on the occupation.

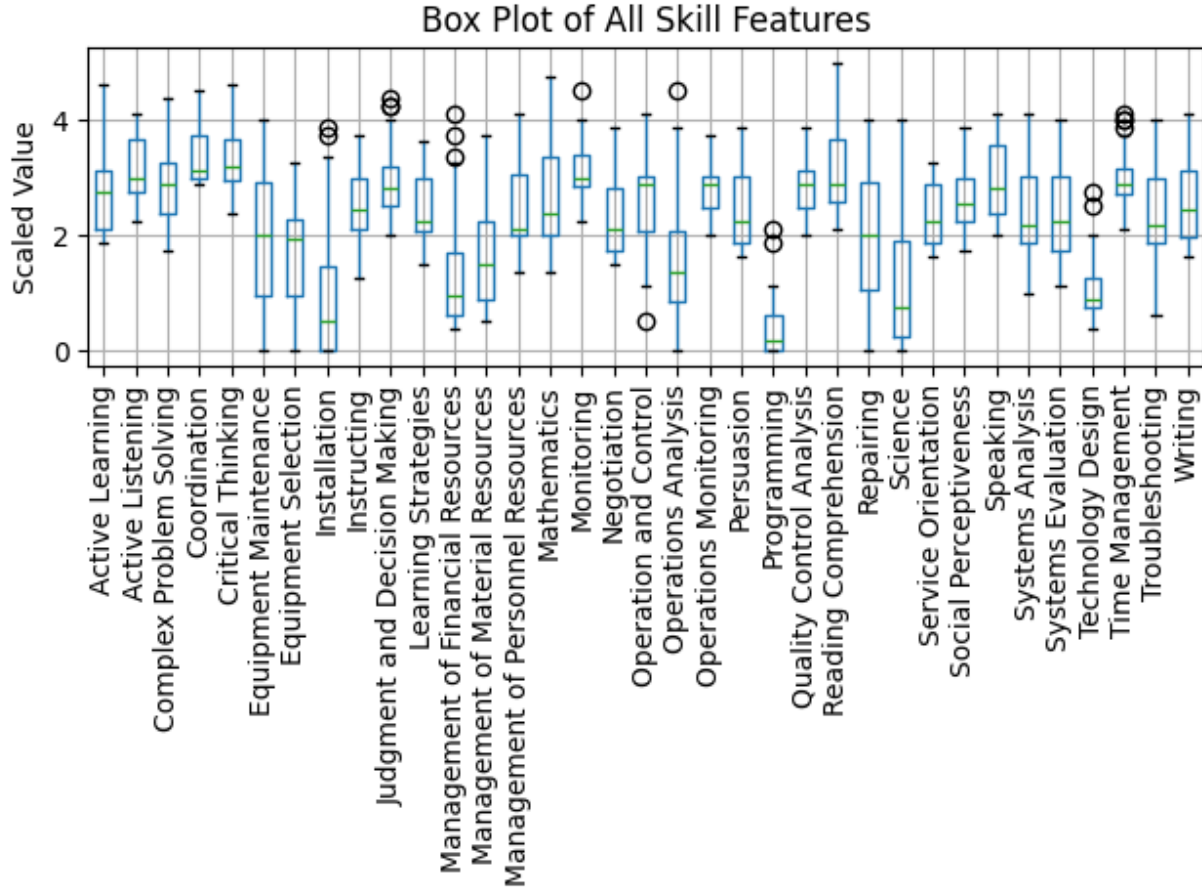


Figure 5: Box Plot of All Filtered Skill Features

The figure above presents the distribution of all skill features using a box plot. Each box represents the interquartile range of a skill, with the median indicated by the central line and whiskers extending to the range of non-outlier values.

From the visualization, it can be stated that many features are concentrated near lower scaled values, with relatively narrow interquartile ranges. This indicates that a large number of skills are either minimally represented or not consistently required across all occupations. In the additional figure below, the bar graph outlines the distribution of skill-sets, where the y axis is the mean value or skill mean value of the skill-sets that appear in the filtered dataset. The filtered dataset is determined as the 30 filtered features from the 62,000 row skill-set ONET database. Additionally, the x axis is the label of the given skill-set. This label is represented as a string.

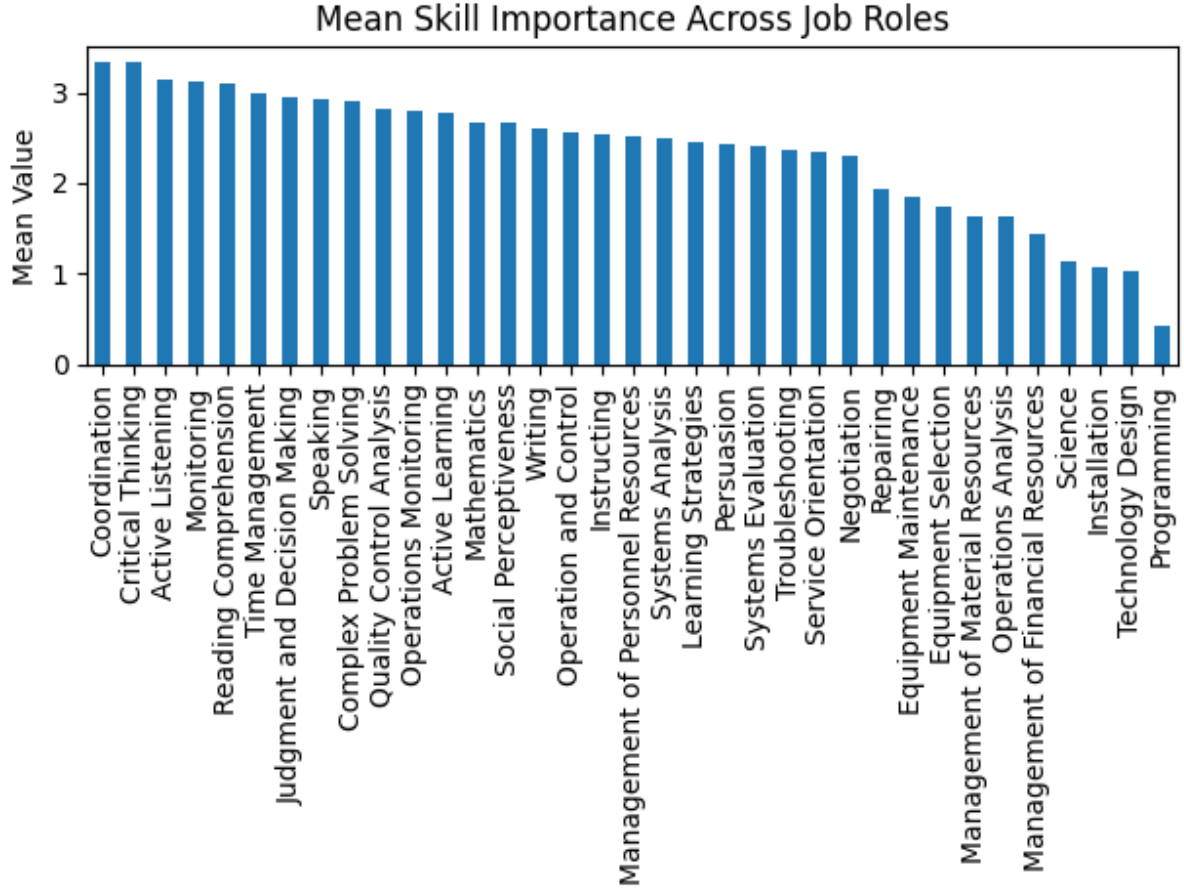


Figure 6: Bar Chart of Mean Skillset Values

6 Implementation

The system is implemented in Python using a modular pipeline that integrates data pre-processing, feature engineering, similarity computation, and dimensionality reduction. Core libraries include `pandas` and `numpy` for data manipulation, `pdfplumber` for text extraction, `scikit-learn` for scaling, similarity computation, and PCA, and `plotly` for visualization.

6.1 Dataset Construction and Filtering

The foundation of the system is the O*NET Skills dataset, which provides standardized numerical representations of skill importance across occupations. Each entry in the dataset contains an O*NET-SOC code, a skill name (Element Name), and a corresponding numerical value (Data Value).

To ensure consistency, only entries corresponding to the “Level” scale are retained:

$$D' = \{(c, s, v) \mid \text{Scale Name} = \text{Level}\} \quad (1)$$

where c denotes the occupation code, s denotes a skill, and v represents its importance value.

In order to focus the analysis on a relevant subset of occupations, the dataset is filtered using a predefined list of O*NET-SOC codes, chosen at random with preferences with that of construction related roles. Formally, let $\mathcal{C}$ denote the selected set of occupation codes. The filtered dataset is given by:

$$D_{\mathcal{C}} = \{(c, s, v) \in D' \mid c \in \mathcal{C}\} \quad (2)$$

This filtering step is critical because it constrains the feature space to a domain-specific subset.

6.2 Feature Matrix Construction

The filtered dataset is transformed into a structured matrix representation using a pivot operation. Let M denote the number of job roles and n denote the number of skills. The resulting matrix is:

$$X \in \mathbb{R}^{M \times n} \quad (3)$$

where each row corresponds to an occupation and each column corresponds to a skill. Each entry X_{ij} represents the importance of skill j for occupation i .

Missing values are replaced with zero:

$$X_{ij} = \begin{cases} v_{ij}, & \text{if value exists} \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

To reduce noise and remove uninformative features, a variance threshold is applied:

$$\text{Var}(X_{.j}) > \tau \quad (5)$$

where $\tau = 0.1$. This results in a reduced feature space of size n' , where $n' < n$.

6.3 Feature Scaling

Because the associated skill values may vary in magnitude across features, Min-Max normalization is applied for scaling purposes:

$$X'_{ij} = \frac{X_{ij} - \min(X_{.j})}{\max(X_{.j}) - \min(X_{.j})} \quad (6)$$

This transformation maps all features into the range $[0, 1]$, preventing any single feature from dominating similarity computations.

The scaling parameters are learned from the job dataset and subsequently applied to all resume vectors to maintain consistency.

6.4 Resume Ingestion and Preprocessing

Resumes are ingested from a directory containing PDF and text files. PDF files are processed using PDFPlumber, which extracts text on a per-page basis and concatenates the results.

The extracted text is then normalized using a preprocessing function:

- Conversion to lowercase
- Removal of non-alphanumeric characters
- Normalization of whitespace

Let T denote the raw text and $\tilde{T}$ denote the cleaned text:

$$\tilde{T} = f(T) \quad (7)$$

where f represents the cleaning function.

6.5 Feature Extraction from Resumes

To map resumes into the same feature space as job roles, a keyword-based extraction method is used. Each skill s_j is associated with a set of keywords derived from its name:

$$K_j = \{k_{j1}, k_{j2}, \dots, k_{jm_j}\} \quad (8)$$

The feature value for each skill is computed as the total number of keyword occurrences:

$$x_j = \sum_{k \in K_j} \text{count}(k, \tilde{T}) \quad (9)$$

This produces a resume feature vector:

$$\mathbf{x}_{res} \in \mathbb{R}^{n'} \quad (10)$$

which is aligned with the job feature matrix.

6.6 Similarity Computation in Original Space

Cosine similarity is used to measure the alignment between resume and job vectors:

$$\text{sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \quad (11)$$

For each resume vector $\mathbf{x}_{res}$, similarity is computed against all job vectors:

$$s_i = \text{sim}(\mathbf{x}_{res}, \mathbf{x}_i) \quad (12)$$

The system ranks occupations based on these scores and returns the top matches.

6.7 Principal Component Analysis

To reduce dimensionality and enable visualization, Principal Component Analysis (PCA) is applied to the scaled job matrix X' . PCA computes an orthogonal transformation:

$$Z = X'W \quad (13)$$

where $W \in \mathbb{R}^{n' \times k}$ contains the top k eigenvectors of the covariance matrix.

In this implementation, $k = 3$, resulting in a three-dimensional representation:

$$Z \in \mathbb{R}^{M \times 3} \quad (14)$$

Each principal component corresponds to a direction of maximum variance, determined by the eigenvalues of the covariance matrix:

$$\Sigma W = W\Lambda \quad (15)$$

where Λ is a diagonal matrix of eigenvalues.

The explained variance ratio is computed as:

$$\text{EVR}_i = \frac{\lambda_i}{\sum_j \lambda_j} \quad (16)$$

This provides a measure of how much information is retained in the reduced space.

Resume vectors are projected using the same transformation:

$$\mathbf{z}_{res} = \mathbf{x}_{res}W \quad (17)$$

allowing for both jobs and resumes to reside in a shared latent space.

6.8 Similarity in PCA Space

Cosine similarity is recomputed using the transformed vectors:

$$s_i^{(PCA)} = \text{sim}(\mathbf{z}_{res}, \mathbf{z}_i) \quad (18)$$

This allows comparison between similarity results in the original and reduced spaces.

6.9 Visualization

The final stage of the system involves visualization of the PCA-transformed data. Job roles and resumes are combined into a single dataset:

$$Z_{all} = \{Z_{jobs}, \mathbf{z}_{res}\} \quad (19)$$

A 3D scatter plot is generated using Python’s Plotly, with axes corresponding to the first three principal components (PC1, PC2, PC3). Job points are displayed as smaller markers, while the resume is emphasized using a larger marker size.

This visualization provides an intuitive geometric interpretation of similarity relationships and allows users to observe how resumes align with clusters of occupations.

6.10 Explained Variance Analysis

The effectiveness of the PCA transformation can be evaluated using the explained variance ratio, which quantifies how much of the total variance in the original feature space is captured by each principal component.

In this implementation, the explained variance ratios are:

$$[0.6596, 0.1627, 0.0443] \tag{20}$$

These values correspond to the first three principal components (PC1, PC2, and PC3), respectively.

6.10.1 Interpretation of Individual Components

Each value represents the proportion of total variance explained by a principal component:

- **PC1 (65.96%):** The first principal component captures the majority of the variation in the dataset. This indicates that a single dominant pattern governs most of the differences between job roles. In the context of the dataset, this likely reflects a major distinction such as trade-oriented versus planning-oriented roles.
- **PC2 (16.27%):** The second principal component captures an additional, independent source of variation. This component represents a secondary structure in the data, such as differences between technical and non-technical roles.
- **PC3 (4.43%):** The third principal component contributes only a small amount of additional information, capturing more subtle variations between occupations.

6.10.2 Total Explained Variance

The total variance captured by the three components is given by:

$$0.6596 + 0.1627 + 0.0443 = 0.8666 \quad (21)$$

Thus, approximately **86.66%** of the total variance in the original high-dimensional feature space is preserved in the three-dimensional PCA representation.

6.10.3 Implications for Dimensionality Reduction

This result indicates that PCA achieves substantial dimensionality reduction while retaining most of the important structure of the data. Specifically:

- The original feature space contains 30 dimensions (skills).
- PCA reduces this to 3 dimensions.
- Despite this reduction, over 86% of the information is preserved.

This confirms that the dataset contains a high degree of redundancy, meaning that many skill dimensions are correlated and can be represented using a smaller number of underlying factors.

6.10.4 Connection to Similarity Results

The explained variance distribution provides insight into the behavior observed in the similarity results.

Because PC1 alone captures approximately 66% of the total variance, most job vectors are strongly aligned along this dominant direction. When data is projected into PCA space, this results in vectors becoming more collinear, which in turn increases cosine similarity scores.

This explains why similarity values in PCA space are significantly higher than those in the original feature space. The dimensionality reduction compresses variation along less important dimensions, effectively reducing the angular separation between vectors.

7 Results Visualized

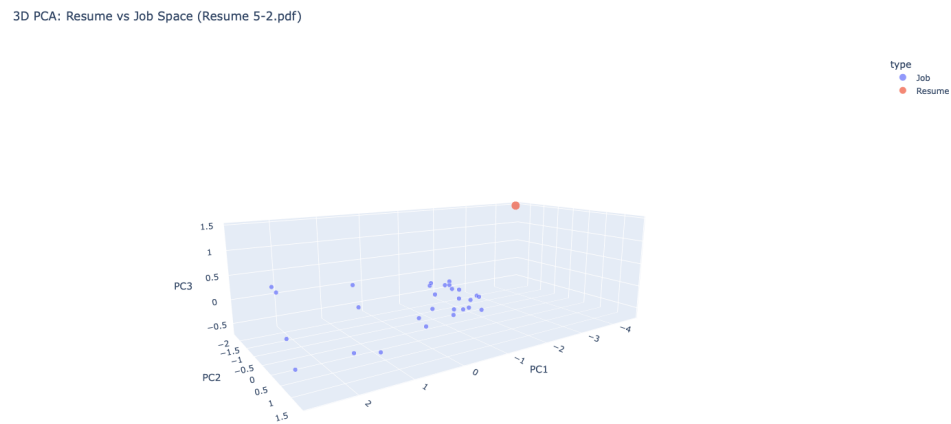


Figure 7: 3D PCA Projection of Resume 5-2 in Job Skill Space

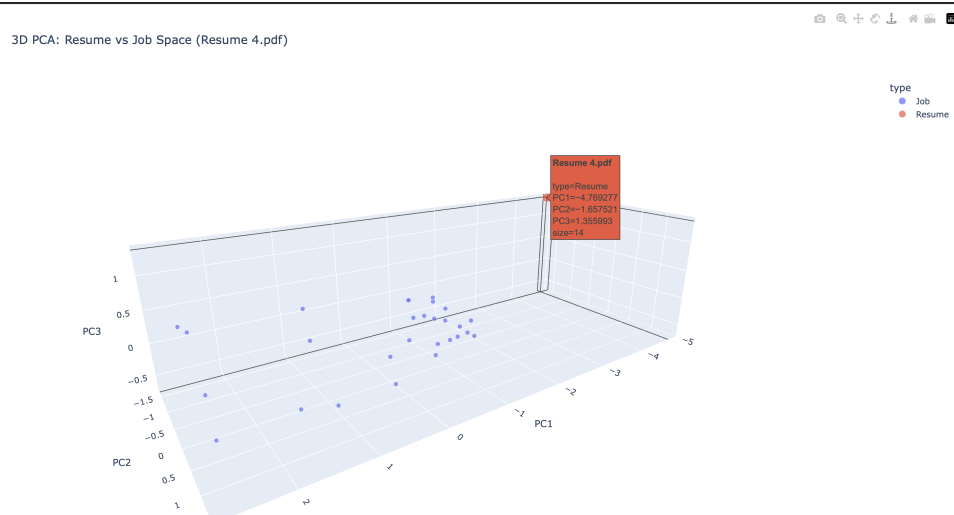


Figure 8: 3D PCA Projection of Resume 4 in Job Skill Space

The PCA plots show that job roles form a compact cluster near the origin, while both resumes lie clearly outside this region. This indicates that neither resume closely aligns with the overall job skill distribution.

In both cases, the separation is most pronounced along the third principal component (PC3), where the resumes deviate significantly from the job cluster. Resume 5-2 shows some alignment in the primary components (PC1 and PC2) but diverges along PC3, suggesting partial overlap with common skill patterns but differences in more specific dimensions.

Resume 4 is more strongly separated, with deviation across both PC1 and PC3, indicating a broader mismatch in skill representation.

Overall, both resumes behave as outliers in the PCA space, with their distances, especially along PC3. These distances are influenced by their differences between their extracted features and the structured job skill profiles.

8 U-MAP Visualization and Interpretation

UMAP Job Space

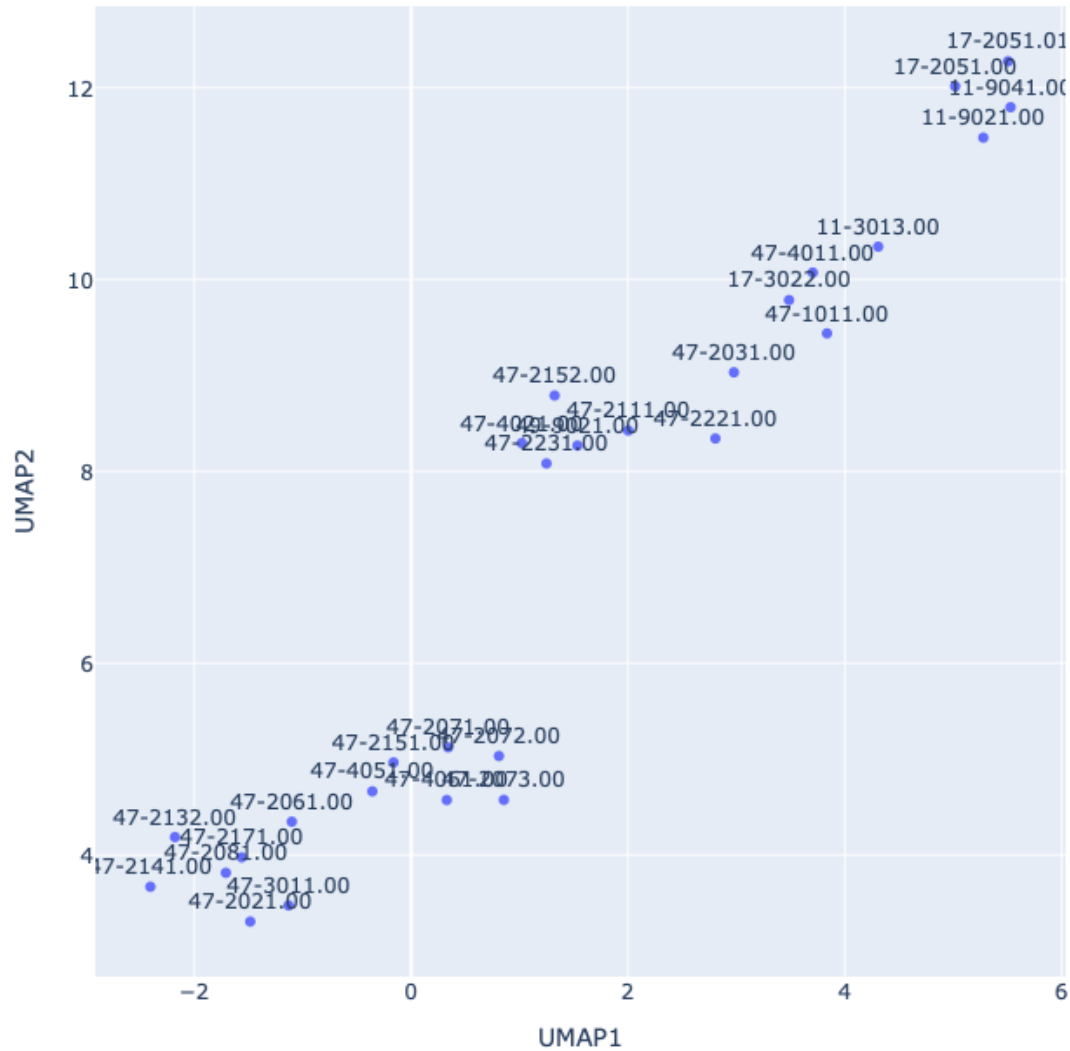


Figure 9: UMAP Projection of Job Skill Space

Uniform Manifold Approximation and Projection (UMAP) is used to visualize the structure of the job skill dataset in a two-dimensional space while preserving both local and global

relationships between observations. Unlike PCA, which is a linear dimensionality reduction technique, UMAP captures non-linear patterns in the data, making it particularly effective for identifying clusters and the intrinsic structure within high-dimensional feature spaces.

In the visualization above, each point represents a job role, labeled by its corresponding O*NET-SOC code. The plot showcases clusters with job roles with distinct groupings along a diagonal trajectory from the lower-left to the upper-right region of the space. This gradient suggests a continuous skill similarity, where occupations that are positioned closer together share more similar skill profiles.

The axes in this 2-Dimensional projection are UMAP1 and UMAP2. Where both axes acts as a coordinate system. This allows for each ONET job role to be graphed in a lower dimensional space. The location of each job role is dependent on the similarity of it's neighbors. Thus, the formation of clusters are existent. These points in this 2-Dimensional space are also derived from the learned embeddings from the $n = 32$ number of features or dimensions in the original dataset.

In the lower-left region of the plot, job roles are tightly grouped, where a set of occupations with highly similar and potentially overlapping skill requirements. These are likely roles with more standardized or narrowly defined skill sets. Moving toward the center of the plot, the density of points remains relatively high. This suggests that a broad grouping of occupations share common foundational skills.

An important observation is that UMAP produces more clearly defined clusters compared to PCA. While PCA primarily emphasizes directions of maximum variance, UMAP focuses on preserving neighborhood relationships and low dimensional embedding, in tighter groupings of similar occupations. This makes it easier to visually identify clusters of related job roles.

9 Limitations

While the proposed system provides a structured and interpretable approach to resume-to-job matching, it has several limitations that affect its accuracy, generalization, and robustness. These limitations arise primarily from the reliance on handcrafted features, keyword-based extraction, and classical dimensionality reduction techniques.

The most significant limitation is the system’s dependence on predefined keyword dictionaries. Feature extraction relies on matching exact or near-exact terms within the resume text. As a result, the system may fail to recognize semantically equivalent phrases that are not explicitly included in the keyword list. For example, a candidate describing experience as “constructing residential buildings” may possess carpentry-related skills, but these may not be captured unless the relevant keywords are predefined. This limits the system’s ability to generalize across diverse writing styles and reduces its effectiveness in capturing nuanced language.

A related limitation is the lack of contextual understanding. The system treats all keyword occurrences equally, without considering their context or importance within the resume. For instance, a brief mention of a skill is weighted the same as extensive experience in that area. Additionally, the system cannot distinguish between different levels of expertise, such as beginner versus advanced proficiency. This simplification can lead to inaccurate representations of a candidate’s true qualifications.

Another limitation arises from the use of raw frequency counts for feature values. Although Min-Max scaling normalizes these values, it does not fully address the issue of disproportionate keyword usage. Longer resumes or those with repetitive phrasing may produce inflated feature values, introducing bias into similarity calculations. This can result in higher similarity scores that do not necessarily reflect stronger alignment with job requirements.

The application of cosine similarity also introduces constraints. While cosine similarity is effective for measuring vector alignment, it assumes that the feature space adequately captures all relevant dimensions of similarity. If important skills are missing or underrepre-

sented in the feature set, the similarity metric may produce misleading results. In addition, cosine similarity does not account for hierarchical relationships between skills or dependencies among features.

PCA, while useful for the visualization portion of this project, presents further limitations. PCA reduces dimensionality by maximizing variance, not by optimizing for matching accuracy. Consequently, information that is important for distinguishing between job roles may be lost during projection. The loss of relational aspects is an important consideration when deciding on model compression. This can lead to inconsistencies between similarity results in the original feature space and those in the reduced space.

The system is also limited by the quality and scope of the job dataset. Because the feature scaling and similarity comparisons are based on this dataset, any bias or imbalance within the job data will directly affect the results.

The system does not incorporate advanced natural language processing techniques such as semantic embeddings or contextual language models. As a result, it lacks the ability to capture deeper relationships between words and phrases. Modern approaches using transformer-based models can understand meaning at a much higher level, enabling more accurate and flexible matching. The absence of such techniques limits the system's performance compared to state-of-the-art solutions.

10 Recommendations

Outlined in this section are the recommendations to the audience that are adopting the Resume-to-job matching model proposed in this report.

First, the feature extraction process should be extended beyond simple keyword matching. While the current dictionary-based approach provides an ease of understanding structurally, it lacks the ability to capture semantic meaning. To address this, it is recommended to incorporate Term Frequency–Inverse Document Frequency (TF-IDF) weighting. TF-IDF

reduces the impact of commonly occurring terms while emphasizing that of more informative and distinctive words.

Second, the system would benefit significantly from the integration of semantic embeddings. Modern natural language processing models, such as Word2Vec, GloVe, or transformer-based embeddings, can capture meaning and relationships between words. By representing resumes and job descriptions as dense embedding vectors, the system could recognize semantically similar phrases even when exact keywords are not present.

Additionally, feature weighting should be introduced to reflect the relative importance of different skills. In the current system, all features are treated equally, which may not align with real-world hiring priorities. For example, safety compliance or project management may be more critical than general labor skills in certain roles. Assigning weights to features based on domain knowledge or data-driven methods would improve the relevance of similarity rankings.

Another important recommendation is to refine the use of PCA. While PCA is effective for visualization, the results show that it can distort similarity rankings and produce overly generalized matches. Therefore, PCA should be retained primarily as an exploratory and visualization tool rather than as a core component of the matching process.

11 Conclusion

This paper presents a structured and interpretable approach to resume-to-job matching using feature extraction, cosine similarity, and Principal Component Analysis (PCA). By transforming unstructured resume text into numerical feature vectors, this system enables direct comparison between candidate profiles and job roles. The use of cosine similarity provides a useful mechanism for ranking job matches based on alignment in a shared feature space, while PCA offers additional insight into the structure and relationships within the data.

One of the primary contributions of this work is the demonstration that relatively simple, classical data science techniques can produce meaningful and interpretable results in a complex domain such as resume matching. The feature engineering approach, based on specific keyword groupings, allows resumes to be represented in a structured format that preserves important skill information.

The experimental results indicate that cosine similarity in the original feature space produces the most reliable and interpretable job matches. Resumes with strong and clearly defined skill signals were matched with according O*NET occupations. Thus, this demonstrates the effectiveness of the feature representation, especially when the underlying data is aligned. However, resumes with sparse or ambiguous keyword representation produced weaker similarity scores and less precise matches.

Principal Component Analysis was shown to be valuable for visualization and exploratory analysis. The reduction of high-dimensional feature vectors into a three-dimensional space enables intuitive understanding of how resumes and job roles are positioned relative to one another. Clustering patterns observed in the PCA space provide insight into shared skill structures and occupational groupings. However, the results also demonstrate that PCA may reduce matching accuracy due to information loss during dimensionality reduction. In several cases, PCA produced inflated similarity scores and more generalized job matches, thus indicating that it prioritizes dominant variance.

The system developed in this work serves as a strong baseline for resume-to-job matching. Future work can build upon this framework by incorporating more sophisticated natural language processing techniques, such as semantic embeddings and contextual language models, which are capable of capturing deeper relationships between words and phrases.

Additionally, improvements in feature weighting, normalization, and dataset construction can further enhance performance. Integrating larger datasets, such as other standardized occupational skill databases, would provide more reliable and consistent feature values.

12 Work Cited

References

- [1] Jsvine. *pdfplumber*. GitHub.
<https://github.com/jsvine/pdfplumber>
- [2] Scikit-learn Developers. *sklearn.decomposition.PCA*. Scikit-learn.
<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>
- [3] Van der Maaten, Laurens, and Geoffrey Hinton. *Visualizing Data using t-SNE*. arXiv, 2008.
<https://arxiv.org/abs/1301.3781>
- [4] Northeastern University. *Final Project Data*. Canvas.
<https://northeastern.instructure.com/courses/246320/files/folder/Final>
- [5] McInnes, Leland, John Healy, and James Melville. *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. UMAP Documentation.
<https://umap-learn.readthedocs.io/en/latest/>
- [6] National Center for O*NET Development. *O*NET Database*. O*NET Resource Center.
<https://www.onetcenter.org/database.html#individual-files>
- [7] University of Maine. *Construction Engineering Technology*. University of Maine Admissions.
<https://umaine.edu/admissions/find-a-major/construction-engineering-technology/>
- [8] University of Maine at Augusta. *Engineering Pathways Program*. University of Maine at Augusta.
<https://www.uma.edu/academics/programs/engineering-pathways/>

- [9] A. H. Grover, Inc. *Employment*. A. H. Grover, Inc.
<http://ahgrover.com/employment/>